

Cómo trabajar con repositorios en Git

10 ejercicios prácticos — de los conceptos básicos a un flujo de trabajo colaborativo

DOCUMENTO DE REFERENCIA

Fastbook 02 · Core Desarrollo

Estos ejercicios repasan qué es un repositorio, Git frente a SVN, los comandos principales de Git y las herramientas gráficas para gestionarlos.

Fácil (1-3)

Fácil-Medio (4-7)

Medio (8-10)

1 Qué es un repositorio

FÁCIL

Explica con tus propias palabras qué es un repositorio de código y enumera al menos dos ventajas de usarlo en un proyecto en equipo.

2 Git frente a SVN

FÁCIL

Completa la siguiente tabla con al menos tres diferencias entre Git y SVN, indicando si el control de versiones es distribuido o centralizado en cada caso.

3 Emparejar comandos con su función

FÁCIL

Relaciona cada comando con lo que hace:

- `git init`
- `git status`
- `git add <archivo>`
- `git commit -m "mensaje"`

Opciones: (a) confirma los cambios con un mensaje descriptivo; (b) crea el subdirectorio `.git` con el esqueleto del repositorio; (c) empieza a trackear un archivo; (d) muestra el estado actual de la rama.

4 Primer flujo de trabajo

FÁCIL-MEDIO

Ordena estos pasos para inicializar un repositorio y subir el primer cambio a un remoto llamado `origin` en la rama `main`: `git commit -m "primer commit"`, `git init`, `git push origin main`, `git add .`

5 Crear y cambiar de rama

FÁCIL-MEDIO

Escribe el comando que crea una nueva rama llamada `feature-login` a partir de la rama actual y te sitúa automáticamente en ella. Después, escribe el comando para listar todas las ramas locales y remotas.

6 git fetch vs git pull

MEDIO

Explica la diferencia entre `git fetch` y `git pull`, y describe un escenario en el que preferirías usar `fetch` antes que `pull`.

7 Deshacer cambios: reset vs revert

MEDIO

Tienes un commit ya subido al repositorio remoto que introdujo un error. Explica por qué en este caso es más adecuado usar `git revert <hash_commit>` que `git reset --hard HEAD`, y qué diferencia hay en el historial resultante.

8 Ramas remotas eliminadas

MEDIO

Un compañero ha eliminado una rama remota que tú todavía ves en tu repositorio local. ¿Qué comando ejecutarías para sincronizar tu copia local y que deje de mostrarse? Explica qué hace exactamente ese comando.

9 Flujo colaborativo en la misma rama

MEDIO

Dos desarrolladores trabajan sobre la misma rama `feature-pagos`. Uno de ellos ya ha subido cambios al remoto. Describe, en orden, la secuencia de comandos que debería usar el segundo desarrollador para bajar esos cambios, integrarlos con los suyos y subir el resultado sin sobrescribir el trabajo de su compañero.

10 Comparar herramientas gráficas

MEDIO

Un equipo trabaja con Windows, macOS y Linux indistintamente. Compara SourceTree y GitKraken según lo visto en el documento y justifica cuál recomendarías para ese equipo, teniendo en cuenta la compatibilidad multiplataforma.

Soluciones orientativas

1. Un repositorio es un lugar donde se almacena el código y su historial de cambios. Ventajas: trabajo colaborativo/en paralelo y seguridad del código (copias de seguridad, recuperación de versiones anteriores).
2. Por ejemplo: Git es distribuido y SVN centralizado; en Git cada copia local tiene el historial completo, en SVN solo el repositorio central lo tiene completo; Git permite trabajar sin conexión salvo para sincronizar, SVN requiere conectividad para casi todo.
3. `git init` → (b); `git status` → (d); `git add` → (c); `git commit -m` → (a).
4. Orden correcto: `git init` → `git add .` → `git commit -m "primer commit"` → `git push origin main`.
5. `git checkout -b feature-login` crea y cambia de rama en un solo paso. Para listar todas las ramas: `git branch -a`.
6. `git fetch` descarga los cambios del remoto pero no los fusiona con tu rama actual; `git pull` hace un `fetch` + `merge` automático. Conviene usar `fetch` cuando quieres revisar los cambios remotos antes de integrarlos.
7. `git revert` crea un nuevo commit que deshace los cambios, conservando el historial íntegro y sin reescribir commits ya compartidos; `reset --hard` elimina commits del historial local, lo que es peligroso si ya se han subido y otros los han descargado.
8. `git remote prune origin` actualiza el repositorio remoto local eliminando las referencias a ramas remotas que ya no existen en el servidor.
9. 1) `git fetch` para ver los cambios remotos; 2) `git pull` (o `fetch` + `merge`) para integrarlos en su copia local; 3) resolver conflictos si los hay; 4) `git add` y `git commit` de sus propios cambios; 5) `git push origin feature-pagos`.
10. Ambas son multiplataforma (Mac, Windows y Linux), por lo que cualquiera encajaría; GitKraken destaca por su interfaz muy visual y sencilla, mientras que SourceTree es igualmente potente y gratuita para uso personal. La elección puede justificarse por la curva de aprendizaje o la documentación disponible para el equipo.

Practica primero sin mirar las soluciones — es la mejor forma de consolidar lo aprendido.